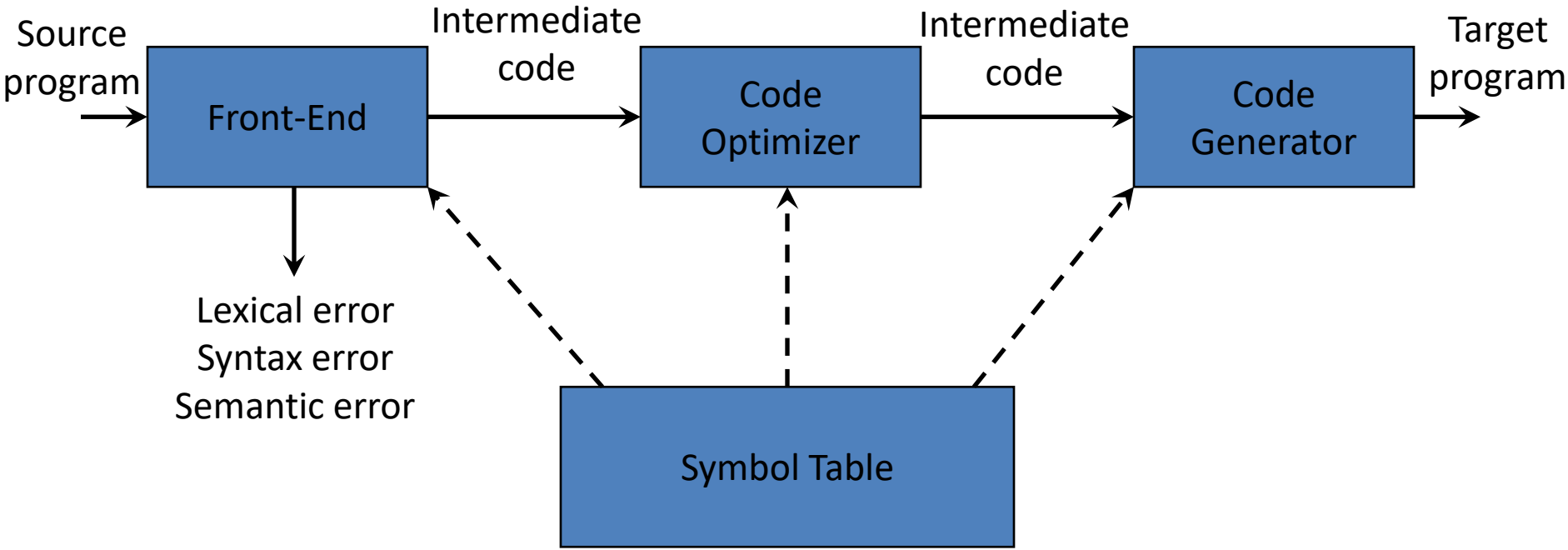
Code Generation

Acknowledgement

- Alfred V Aho, Monica S. Lam, Ravi Sethi, Jeffrey D Ullman- “Compilers- Principles, Techniques and Tools”

Position of a Code Generator in the Compiler Model



Code Generation

- A code generator has three primary tasks:
  - instruction selection
  - register allocation and assignment
  - instruction ordering

Issues in the Design of a Code Generator

- What is the input (form of IR)?
  - May assume input is error-free
- What is the target?
  - Machine (RISC/CISC), absolute / relocatable / assembly
- Instruction selection
- Register allocation
- Evaluation Order

Instruction selection

- The complexity of performing the mapping from lR program into a code sequence is determined by a factors such as
  - level of the IR
    - **High level IR statement** is translated into a sequence of machine instructions using **code templates**
    - produces poor code that needs further optimization
  - nature of the instruction-set architecture
    - **uniformity and completeness** of the instruction set are important factors
    - If the target machine does **not support each data type in a uniform** manner, then each **exception to the general rule** requires special handling
  - desired quality of the generated code
    - Instruction speeds and machine idioms are other important factors
    - **speed and size**
    - IR can be translated into many different code sequences with significant **cost differences**

Instruction selection

- Suppose we translate three-address code

  x:=*y*+*z*

  to:
  ```
  MOV y,R0
  ADD z,R0
  MOV R0,x
  ```
- Then, we translate

  ```
  a:=b+c
  d:=a+e
  ```

  to:
  ```
  MOV a,R0
  ADD b,R0
  MOV R0,a
  MOV a,R0
  ADD e,R0
  MOV R0,d
  ```

  Redundant

Instruction Selection

- Instruction selection is important to obtain efficient code
- Suppose we translate three-address code

$x := y + z$

to:

```
MOV y,R0
ADD z,R0
MOV R0,x
```

```
a:=a+1
```

```
MOV a,R0
ADD #1,R0
MOV R0,a
```

Cost = 6

Better

```
ADD #1,a
```

Cost = 3

Best

```
INC a
```

Cost = 2

Register Allocation

- Efficient utilization of the **limited set of registers** is important to generate good code
- Registers are assigned by
  - *Register allocation* to **select the set of variables** that will reside in registers at a point in the code
  - *Register assignment* to **pick the specific register** that a variable will reside in
- Finding an optimal register assignment in general is NP-complete

Evaluation Order

- The order in which computations are performed can affect the efficiency of the target code
- Some computation orders require **fewer registers** to hold intermediate results than others
- Picking a best order in the general case is a difficult NP-complete problem

The Target Language

- Assembly code as a target language
- A Simple Target Machine Model
  - Instructions: Load, Store, Computation, Unconditional jumps, Conditional jumps
  - Addressing Modes
- Program and Instruction Costs
  - Assume each target-language instruction has an associated cost
  - Cost of an instruction to be **one plus the costs** associated with the **addressing modes** of the operands
  - Addressing modes involving **registers have zero** additional cost, while those involving a **memory location or constant** in them have an **additional cost of one**